

C/C++ Arrays

Ein Array in C++ ist eine zusammenhängende Sequenz von Elementen desselben Datentyps im Arbeitsspeicher (RAM). Seine Größe wird bei der Erstellung festgelegt und ist danach unveränderlich. So kann ein Array beispielsweise mehrere Ganzzahlen verwalten. Der Name eines Arrays entspricht in C++ im Wesentlichen einem Zeiger auf die Startadresse des reservierten Speicherbereichs. Das erste Element befindet sich bei Index 0, da der Index den Versatz (Offset) zur Startadresse angibt. Da alle Elemente direkt hintereinanderliegen und die Datentypgröße bekannt ist, lässt sich die Zieladresse jedes Elements direkt berechnen: $\text{Zieladresse} = \text{Startadresse} + (\text{Index} * \text{Elementgröße in Byte})$

Beispiel: Bei einem int-Array (Größe pro Element: 4 Byte) wird auf das Element bei Index 2 (das 3. Element) zugegriffen. Der Versatz beträgt $2 * 4 \text{ Bytes} = 8 \text{ Bytes}$. Das Programm springt somit von der Startadresse aus um 8 Byte nach vorne, um direkt auf diesen Speicherwert zuzugreifen. Die Deklaration folgt dieser Syntax: `datentyp name[größe];`

Die Größe eines Arrays kann nach der Erstellung nicht mehr geändert werden. Wenn ein Array die Größe 4 besitzt und man versucht, auf den Index 4 zuzugreifen, stürzt das Programm oft nicht sofort ab! C++ erlaubt dir den Zugriff auf diesen verbotenen Speicherbereich. Man liest oder überschreibt dann zufällige Daten (sehr gefährlicher Fehler)

Ein C-Style Array in C++ ist kein Objekt. Es hat kein `.length`-Attribut. Wenn man die Größe zur Laufzeit ermitteln muss, nutzt man den `sizeof`: `sizeof(array) / sizeof(array[0])` (Gesamtbytes durch Byte-Größe eines Elements).

Erstellen eines Arrays und Zuweisung der Werte der einzelnen Elemente:

```
#include <iostream>
int main()
{
    int zahlen[3];
    zahlen[0] = 3;
    zahlen[1] = 5;
    zahlen[2] = 2;
    return 0;
}
```

Erstellen eines Arrays und direkte Zuweisung der Elemente:

```
#include <iostream>
int main()
{
    int zahlen[3] = {10, 20, 30};
    return 0;
}
```

Sizeof verwenden:

```
#include <iostream>
int main()
{
    int array[4] = {5, 9, 8, 0};
    std::cout << sizeof(array) << std::endl; //gibt 16 zurück
    std::cout << sizeof(int) << std::endl; //gibt 4 zurück
    //Anzahl der Elemente ist 4
    std::cout << sizeof(array) / sizeof(int) << std::endl;

    return 0;
}
```

Befüllen eines Arrays mit einer for-Schleife:

```
#include <iostream>
int main()
{
    int array[4];
    for(int i = 0; i < 4; i++)
    {
        array[i] = i;
    }
    return 0;
}
```

Auslesen eines Arrays mit einer for-Schleife:

```
#include <iostream>
int main()
{
    int array[4] = {5, 9, 8, 0};
    for(int i = 0; i < 4; i++)
    {
        std::cout << array[i] << std::endl;
    }
    return 0;
}
```

Sortieralgorithmen:

Nun können Arrays Ganzzahlen in ungeordneter Reihenfolge enthalten, hierfür gibt es verschiedene Sortieralgorithmen. Der einfachste von ihnen ist der Bubblesort-Algorithmus. Dieser Sortiert die Elemente eines Arrays in aufsteigender Reihenfolge, indem er zwei nebeneinanderliegende Elemente miteinander vergleicht. Ist das Element links größer als das rechte Elemente, werden die beiden Elemente vertauscht. Dies wird so lange wiederholt, bis das Array sortiert ist. Der Bubblesort soll nun die Zahlenreihe [2, 4, 1, 3] sortieren. Der Bubblesort-Algorithmus geht nun von links nach rechts paarweise durch die Liste:

Vergleich (2 und 4): Die $2 < 4$. Alles bleibt, wie es ist. Zwischenstand: [2, 4, 1, 3]

Vergleich (4 und 1): Die $4 > 1$. Die beiden Zahlen tauschen die Plätze! Zwischenstand: [2, 1, 4, 3]

Vergleich (4 und 3): Die $4 > 3$. Die beiden Zahlen tauschen! Stand nach dem 1. Durchlauf: [2, 1, 3, 4]

Die größte Zahl (die 4) ist nun wie eine Seifenblase ganz nach rechts gewandert. Dieser gesamte Vorgang wird nun so oft von vorne wiederholt, bis die Reihe perfekt als [1, 2, 3, 4] sortiert ist.

Hier der Code:

```
#include <iostream>
int main()
{
    int array[4] = {2, 4, 1, 3};
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 3; j++)
        {
            if(array[j] > array[j + 1])
            {
                int temp = array[j + 1];
                array[j + 1] = array[j];
                array[j] = temp;
            }
        }

        for(int k = 0; k < 4; k++)
        {
            std::cout << array[k] << std::endl;
        }
        return 0;
    }
}
```

Es gibt viele verschiedene Sortieralgorithmen, wie den Insertionsort, den Selectionsort, den Quicksort oder den Mergesort und viele mehr.